Practical Kubernetes Administration and Troubleshooting

Day 1: Foundations and Cluster Setup

Instructor: Chad M. Crowell

CNCF Ambassador | Kubernetes SIG Contributor | KCD Organizer

About Your Instructor

Chad M. Crowell

- CNCF Ambassador
- Kubernetes SIG Contributor
- KCD Organizer & Speaker
- Site Reliability Engineer
- 17+ years in the industry
- 8+ years teaching DevOps & K8s

Connect

- https://chadmcrowell.com
- GitHub: chadmcrowell

Training content on KubeSkills, Cybr, Pluralsight, and INE

Author of **Acing the Certified Kubernetes Administrator Exam - Second Edition** (acingthecka.com)

Welcome

This program is designed to provide a **practical, hands-on** learning experience focused on real-world Kubernetes administration and troubleshooting.

What to Expect

- Work directly with Kubernetes clusters in **cloud-based environments**
- Guided labs and instructor-led exercises
- Participation, questions, and discussion encouraged
- Share challenges from your own environment

Logistics

- **Duration:** 4 days (7 hours per day, 28 hours total)
- **Format:** Instructor-led, hands-on with cloud lab environments

Day 1 Agenda

Time	Topic
Morning	Containers vs Virtual Machines
	Kubernetes Architecture Overview
	Component Communication & TLS
	Kubernetes Interfaces: CRI, CNI, CSI
Afternoon	Installing and Configuring Kubernetes
	Hands-On Labs

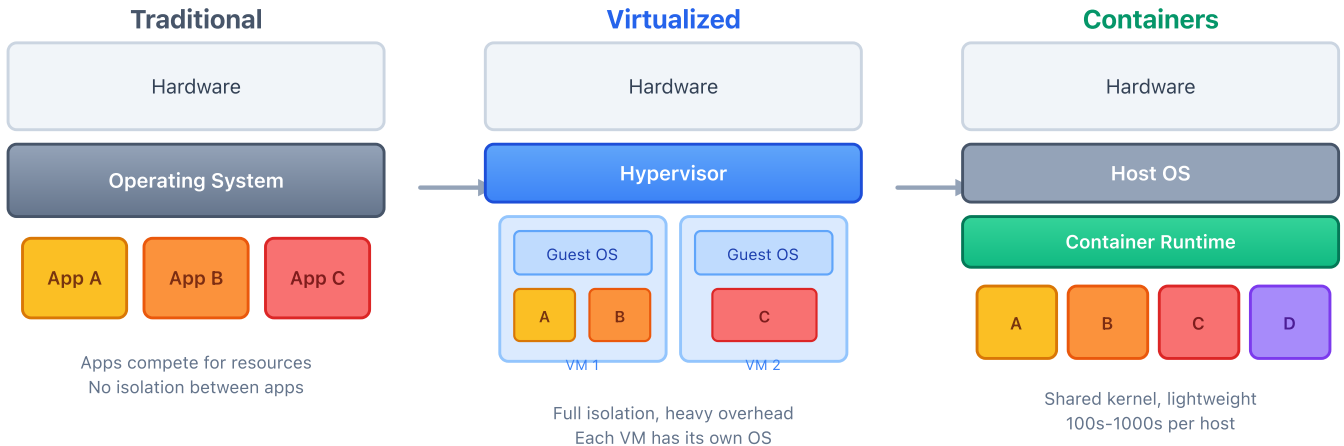
Labs Today

- Provision a cluster on Ubuntu 24.04 (1 control plane + 1 worker)
- Bootstrap with kubeadm and join worker nodes
- Deploy Calico CNI for pod networking
- Validate the cluster and explore core components
- Deploy your first application

Containers vs Virtual Machines

Understanding container orchestration

The Evolution of Application Deployment



- **Traditional:** Apps compete for resources on a single OS
- **Virtualized:** Full OS per VM – isolation but heavy overhead
- **Containers:** Shared kernel, isolated processes – lightweight and fast

Virtual Machines vs Containers

Feature	Virtual Machines	Containers
Isolation	Full OS-level	Process-level (namespaces, cgroups)
Boot time	Minutes	Seconds
Size	Gigabytes	Megabytes
OS	Each VM has its own OS	Share host kernel
Overhead	High (hypervisor + guest OS)	Low (shared kernel)
Density	10s per host	100s–1000s per host
Portability	Tied to hypervisor	Runs anywhere with a container runtime

Why Container Orchestration?

Running a **single container** is easy. Running **hundreds across multiple hosts** is not.

Challenges Without Orchestration

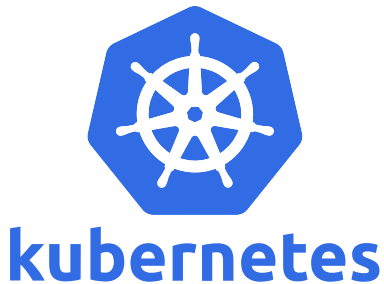
- How do you schedule containers across nodes?
- What happens when a container crashes?
- How do containers find and talk to each other?
- How do you roll out updates without downtime?
- How do you scale up and down based on demand?

What Orchestration Provides

- **Scheduling** – Place workloads on available nodes
- **Self-healing** – Restart failed containers automatically
- **Service discovery** – Built-in DNS and load balancing
- **Rolling updates** – Zero-downtime deployments
- **Scaling** – Horizontal pod autoscaling

Why Kubernetes?

- Open source, originally designed by Google (based on Borg)
- Donated to the **Cloud Native Computing Foundation (CNCF)** in 2015
- The de facto standard for container orchestration
- Massive ecosystem and community



Key Benefits

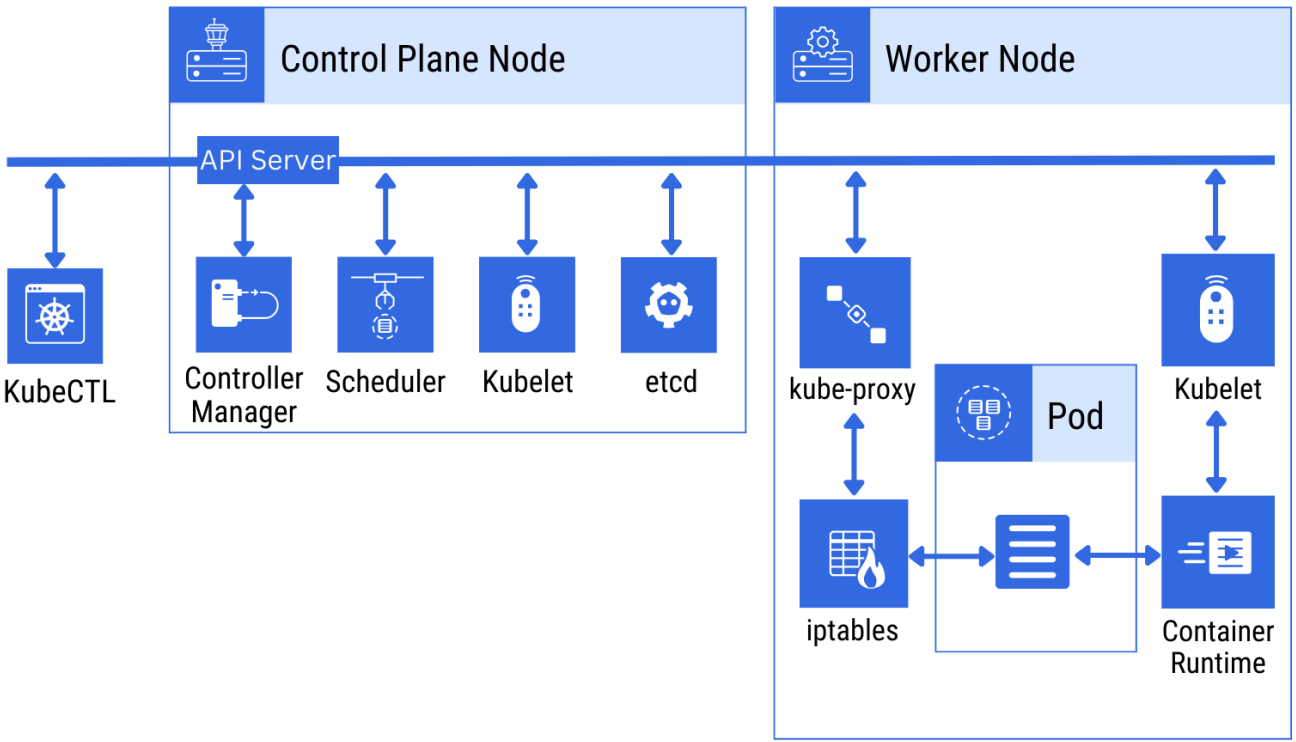
- **Declarative configuration** – describe desired state, K8s makes it happen
- **Portable** – runs on any cloud, on-prem, or hybrid
- **Extensible** – CRDs, operators, plugins for everything
- **Self-healing** – automatically replaces and reschedules failed workloads
- **Scalable** – from a single node to thousands

> "Kubernetes is the Linux of the cloud." – Kelsey Hightower

Kubernetes Architecture

Nodes, pods, control plane, and components

Kubernetes Architecture Overview

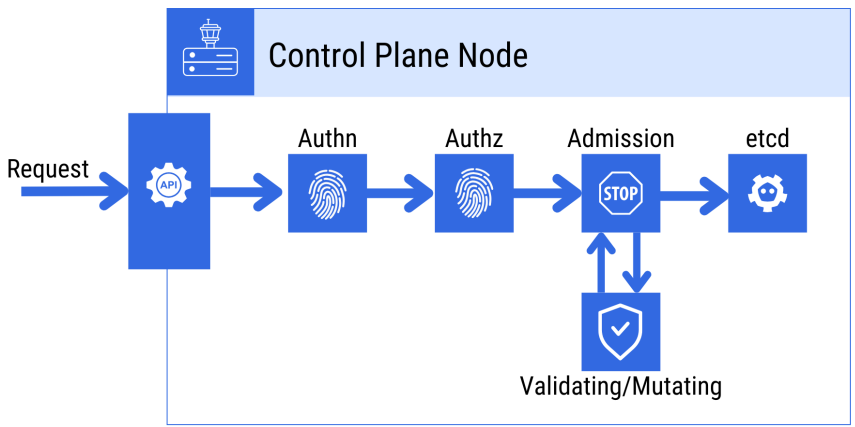


Control Plane Components

The control plane makes global decisions about the cluster.

kube-apiserver

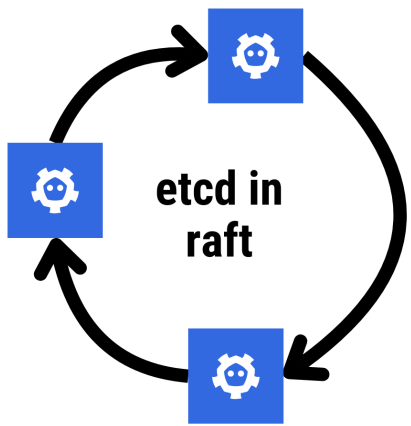
- The front door to the cluster – all communication goes through it
- RESTful API – `kubectl`, dashboards, and other tools talk to this
- Handles authentication, authorization, and admission control



Control Plane Components (cont.)

etcd

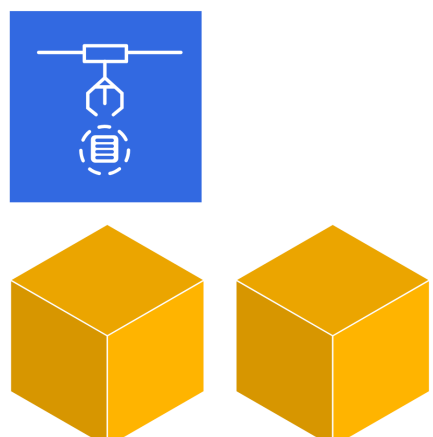
- Distributed key-value store
- Stores **all** cluster state and configuration
- The single source of truth – if etcd is gone, the cluster is gone
- etcd provides strong consistency through the Raft consensus algorithm



Control Plane Components (cont.)

kube-scheduler

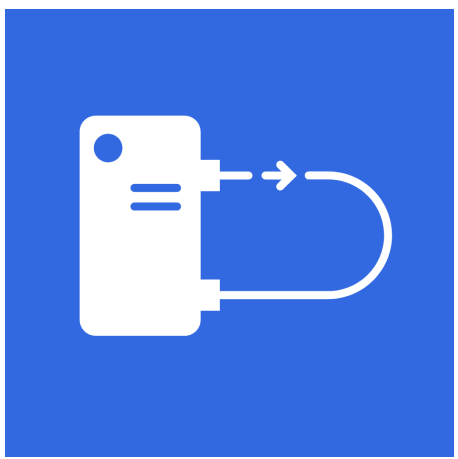
- Watches for newly created Pods with no assigned node
- Selects the best node based on:
  - Resource requirements (CPU, memory)
  - Affinity/anti-affinity rules
  - Taints and tolerations
  - Data locality



Control Plane Components (cont.)

kube-controller-manager

- Runs controller loops that regulate the state of the cluster
- Key controllers:
  - **Node Controller** – monitors node health
  - **Replication Controller** – maintains correct pod count
  - **Endpoints Controller** – populates service endpoints
  - **Service Account Controller** – creates default accounts for namespaces



Worker Node Components

Every worker node runs these components to maintain running Pods.

kubelet

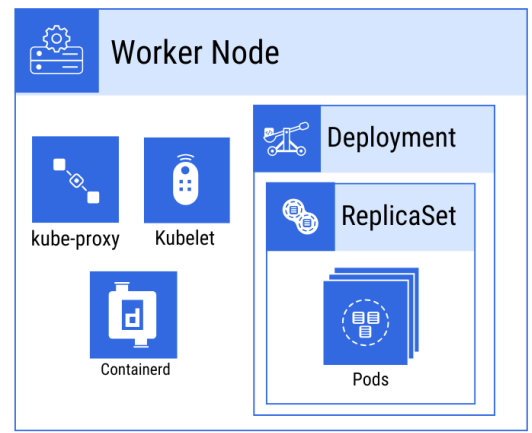
- Agent running on each node
- Ensures containers described in PodSpecs are running and healthy
- Reports node status back to the API server

kube-proxy

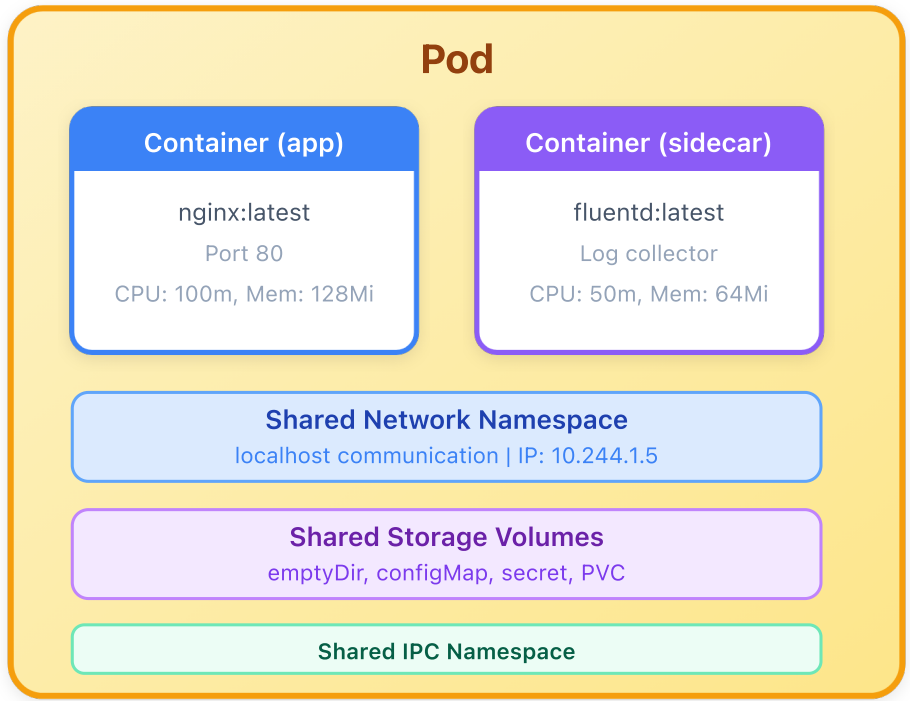
- Network proxy running on each node
- Maintains network rules (iptables/IPVS) for Service communication
- Enables the Service abstraction

Container Runtime

- Software responsible for running containers
- Kubernetes supports any **CRI-compatible** runtime (e.g. **containerd**, CRI-O)



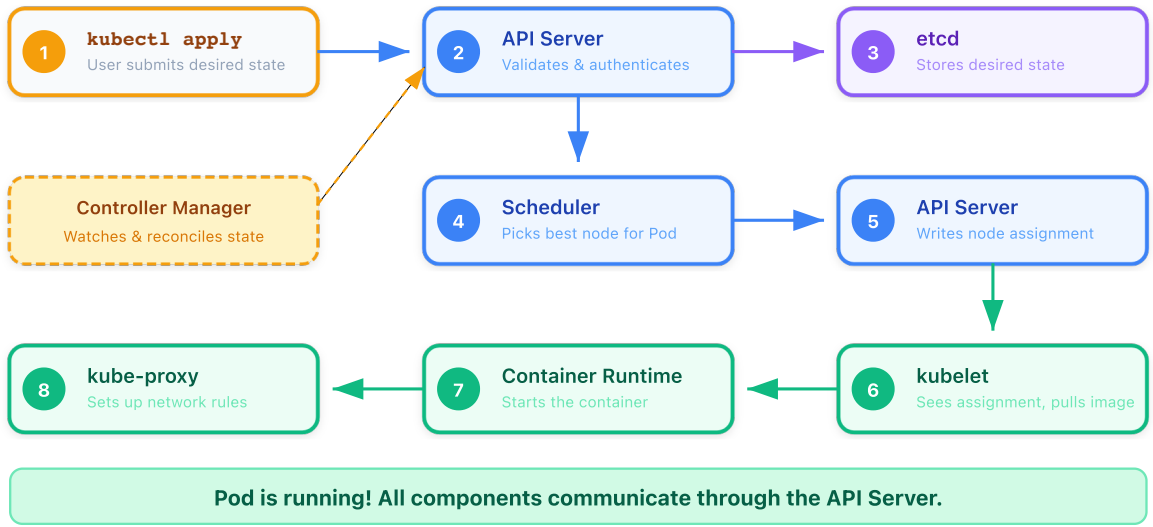
Pods – The Smallest Deployable Unit



Key Facts

- Smallest deployable unit
- 1+ containers per Pod
- Unique IP per Pod
- Containers share network + storage
- Ephemeral by design
- Managed by controllers (Deployment, ReplicaSet)

How Components Work Together



Key insight: The API Server is the only component that talks to etcd. All other components (Scheduler, kubelet, Controller Manager) communicate exclusively through the API Server — it is the single source of truth for the entire cluster.

Securing Component Communication with TLS

All communication between components is encrypted and mutually authenticated using TLS certificates.

Server Certificates

Each component that **serves** an API presents a server certificate to prove its identity.

- **API Server** – serves HTTPS on port `6443`
- **etcd** – serves its API to the API Server
- **kubelet** – serves its own HTTPS API on each node

Client Certificates

Each component that **connects** to another presents a client certificate to authenticate itself.

- **kubelet → API Server** – proves node identity
- **Scheduler → API Server** – proves scheduler identity
- **Controller Manager → API Server** – proves CM identity
- **API Server → etcd** – proves API Server identity
- **API Server → kubelet** – proves API Server identity

> **Mutual TLS (mTLS):** Both sides verify each other's certificates against the cluster CA. This is why `kubeadm init` generates a full PKI under `/etc/kubernetes/pki/` – without it, no component can join the cluster.

Container Runtime Interface (CRI)

CRI is the plugin API that lets the kubelet use **any** container runtime without being compiled against it.

How It Works

- kubelet communicates with the runtime over a **gRPC** socket
- Two services: **RuntimeService** (manage containers) and **ImageService** (manage images)
- kubelet doesn't care *which* runtime — only that it speaks CRI

Common CRI Runtimes

Runtime	Detail
containerd	Default in most distros, graduated CNCF project
CRI-O	Built specifically for Kubernetes, used by OpenShift

> **History:** Kubernetes originally embedded Docker support directly. The **dockershim** was removed in v1.24 — all runtimes now must implement CRI.

Container Network Interface (CNI)

CNI is the standard that defines how **network plugins** configure networking for containers.

How It Works

- When a pod is created, the kubelet asks the container runtime (not directly the CNI plugin) to create the pod.
- The container runtime then creates a network namespace and invokes the CNI plugin with an ADD command
- Assigns an IP address through its IPAM (IP Address Management) plugin
- Sets up routing rules and network policies
- When the pod is terminated, the container runtime issues a DEL command to the CNI plugin, which removes interfaces, releases IP addresses, and cleans up routes.

Fun Fact: CNM and CNI emerged around the same time (2015-2016) as competing visions for container networking. Docker built CNM tightly integrated with its runtime (using libnetwork), while CoreOS and the broader cloud-native community created CNI as a lightweight, runtime-agnostic specification. CNI won the orchestration wars—Kubernetes, Mesos, and OpenShift all adopted it because its plugin model is simpler to implement and doesn't lock you into Docker.

Common CNI Plugins

Plugin	Notes
Calico	L3 networking + network policy, widely adopted
Cilium	eBPF-based, advanced observability + security
Flannel	Simple overlay network, easy to set up
Weave Net	Mesh network with encryption support

> **Key point:** Kubernetes does *not* ship with a CNI plugin. You must install one after cluster initialization — without it, Pods cannot communicate across nodes.

🎬 Deep dive: Kubernetes Networking and CNI

Container Storage Interface (CSI)

CSI is the standard that lets Kubernetes use **any** storage system without built-in driver code.

How It Works

- Storage vendors ship a **CSI driver** as a set of Pods in the cluster
- Kubernetes calls the driver to **provision**, **attach**, and **mount** volumes
- Admins define storage options via **StorageClasses**

Common CSI Drivers

Driver	Notes
AWS EBS CSI	Block storage for EKS
GCE PD CSI	Persistent disks for GKE
Longhorn	acts as the integration layer that translates Kubernetes storage requests (PVCs, StorageClasses) into Longhorn volume operations, enabling dynamic provisioning of replicated volumes distributed across cluster nodes.
Rook-Ceph	a distributed storage system that provides block (RBD), file (CephFS), and object (S3/Swift) storage, while Rook handles all the operational complexity.

> **Key point:** In-tree volume plugins (e.g. `kubernetes.io/aws-ebs`) are being migrated to CSI drivers. New storage integrations are CSI-only.

Installing and Configuring Kubernetes

Cluster deployment with kubeadm on Linode

Cluster Installation Options

Method	Use Case	Complexity
kubeadm	Production-grade self-managed clusters	Medium
Managed K8s (EKS, GKE, AKS, LKE)	Cloud-managed control plane	Low
k3s	Lightweight / edge / IoT	Low
minikube	Local development	Low
kind	CI/CD testing	Low
kOps	AWS-optimized clusters	Medium
Kubespray	Ansible-based deployment	High

Today: kubeadm

- The official Kubernetes cluster bootstrapping tool
- Creates a **production-ready** cluster
- Handles certificates, static pods, and component configuration
- Best way to understand what a cluster *actually* consists of

Lab Overview

Lab 1: Provisioning the Cluster Spin up 3 Ubuntu 24.04 instances on Linode (1 control plane + 2 workers)

Lab 2: kubeadm Cluster Initialization Prepare nodes, initialize the control plane, and join worker nodes

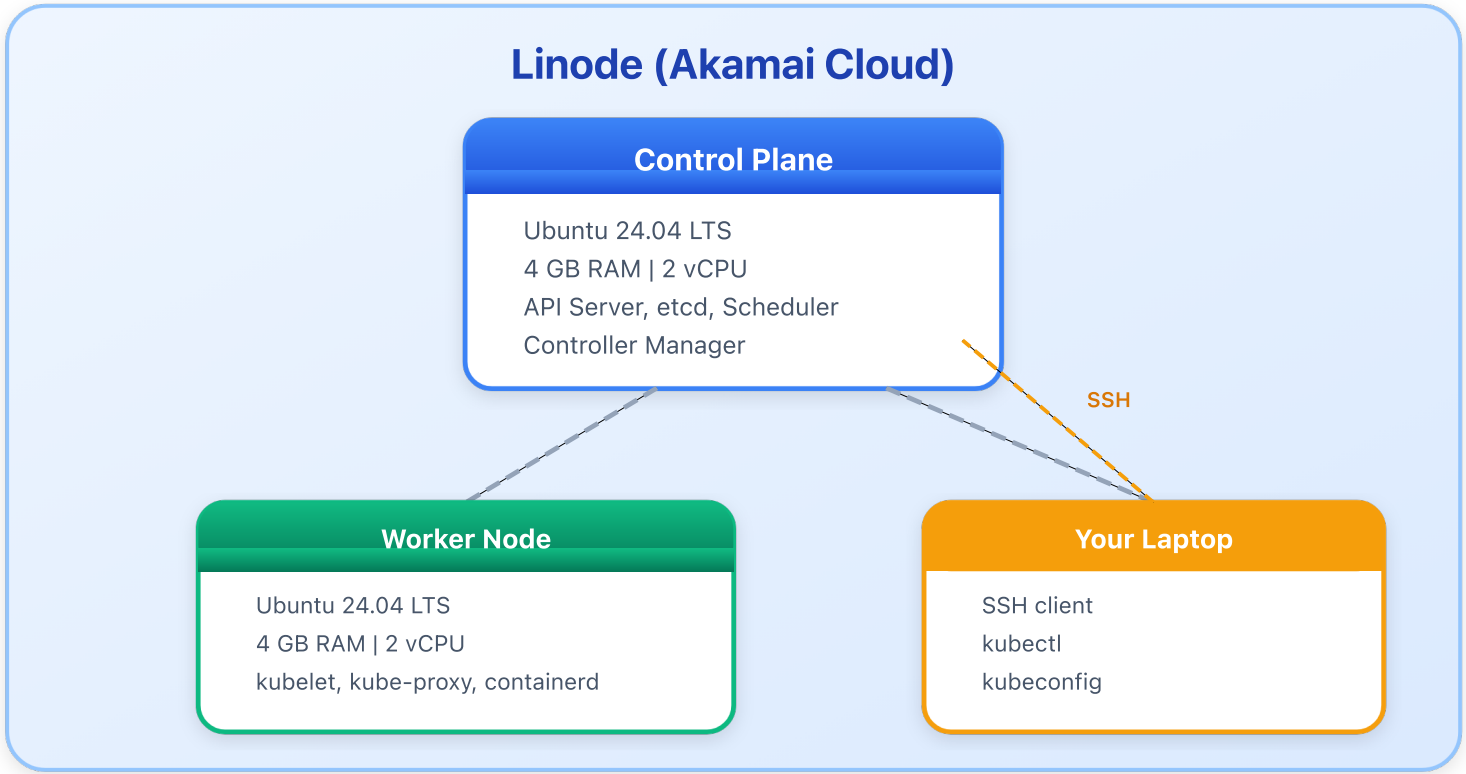
Lab 3: CNI Plugin Installation Deploy Calico for pod networking

Lab 4: Cluster Validation Verify all nodes are Ready and core components are running

Lab 5: Exploring Core Components Inspect etcd, API server, and verify system status

Lab 6: Onboard a Pod Deploy your first application to the cluster

Our Lab Environment



Preparing the Nodes

Before running kubeadm, every node needs:

1. Disable Swap

```
# comment out any lines in /etc/fstab that contain "swap" to prevent swap from being re-enabled on the next boot
sudo swapoff -a
sudo sed -i '/ swap / s/^/#/' /etc/fstab
```

Kubernetes requires swap to be disabled – the kubelet will not start otherwise.

2. Load Required Kernel Modules

```
# Tells Linux to automatically load the overlay (a union filesystem that allows containers to layer read-only image laye
cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF

sudo modprobe overlay
sudo modprobe br_netfilter
```

Preparing the Nodes (cont.)

3. Set Sysctl Parameters

```
# enables iptables processing for bridged IPv4 and IPv6 traffic (bridge-nf-call-iptables and bridge-nf-call-ip6tables) a
cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables  = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward                 = 1
EOF

sudo sysctl --system
```

Preparing the Nodes (cont.)

4. Install containerd

```
sudo apt-get update
sudo apt-get install -y containerd
sudo mkdir -p /etc/containerd
containerd config default | sudo tee /etc/containerd/config.toml
```

Set the cgroup driver to systemd:

```
# enable systemd cgroup driver management for containerd instead of using the default cgroupfs driver
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' /etc/containerd/config.toml
sudo systemctl restart containerd
```

Installing kubeadm, kubelet, and kubectl

Add the Kubernetes apt Repository

```
sudo apt-get install -y apt-transport-https ca-certificates curl gpg

# download k8s GPG signing key
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.34/deb/Release.key | sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-

# create the APT repository configuration file that tells APT where to find k8s packages and which key to use for verifi
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.34/deb/ /' | suc
```

Install and Pin Versions

```
sudo apt-get update
sudo apt-get install -y kubelet kubeadm kubectl
sudo apt-mark hold kubelet kubeadm kubectl
```

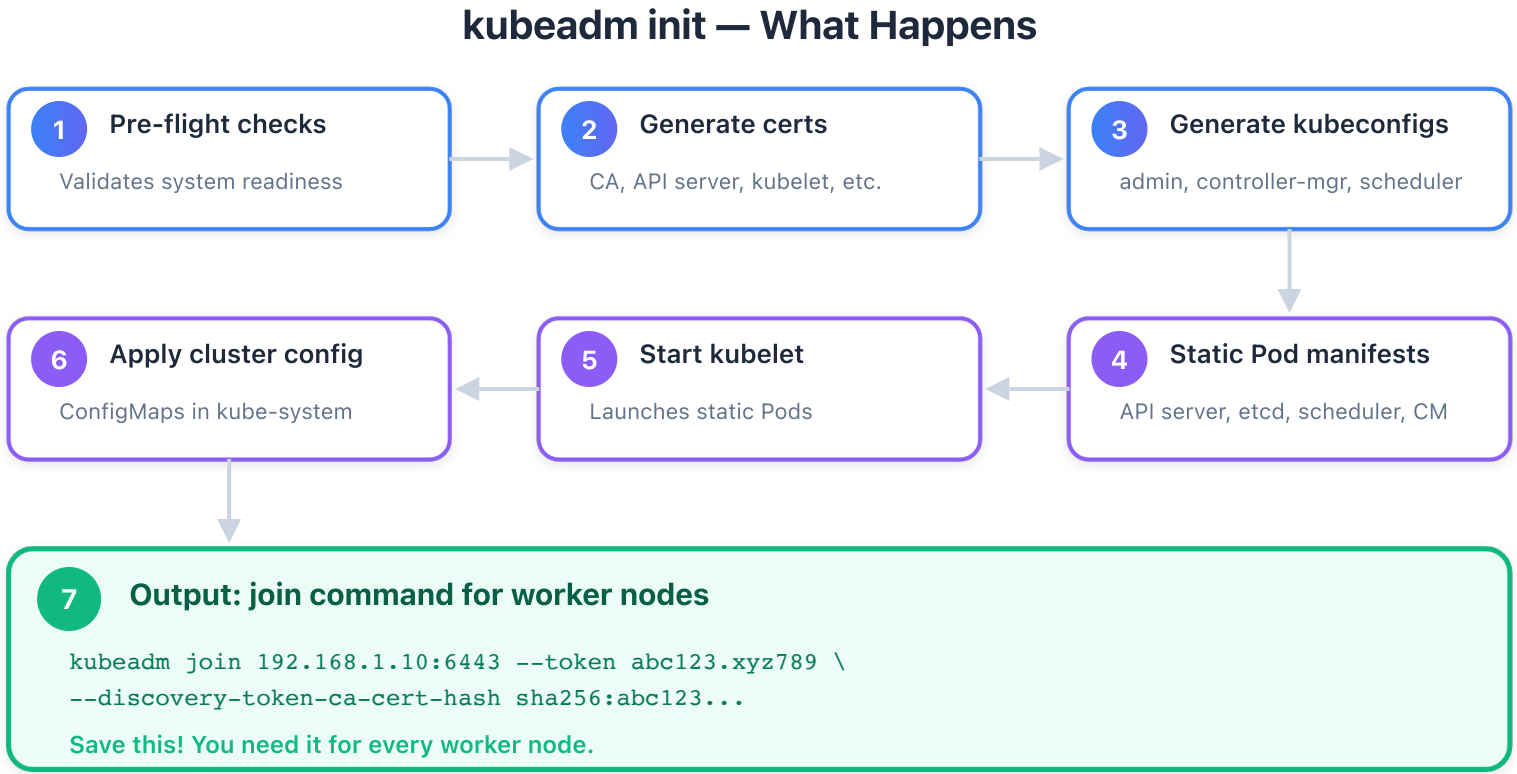
> Pin the versions so `apt-get upgrade` doesn't accidentally break your cluster.

Initializing the Control Plane

Run on the **control plane node only**:

```
sudo kubeadm init \
  --pod-network-cidr=10.100.0.0/16 \
  --kubernetes-version=stable
```

What kubeadm init Does



Configuring kubectl Access

After `kubeadm init`, set up your kubeconfig:

```
mkdir -p $HOME/.kube
sudo cp /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

The `admin.conf` file is a **kubeconfig** — it contains everything kubectl needs to authenticate to the API server.

What’s Inside a kubeconfig?

```yaml
clusters:          # API server address + cluster CA certificate
- cluster:
    certificate-authority-data: <base64-encoded ca.crt>
    server: https://<control-plane-ip>:6443

users:             # client certificate + private key
- user:
    client-certificate-data: <base64-encoded client.crt>
    client-key-data: <base64-encoded client.key>

contexts:          # binds a user to a cluster (and optionally a namespace)
- context:
    cluster: kubernetes
    user: kubernetes-admin
```

kubectl uses **x509 client certificates** to authenticate — it presents `client.crt` to the API server, which verifies it was signed by the cluster CA. The **CN** (Common Name) in the certificate becomes the username and the **O** (Organization) becomes the group.

kubectl Access

Verify the Cluster (on the control plane node)

```
kubectl cluster-info
kubectl get nodes
```

```
NAME    STATUS     ROLES           AGE   VERSION
cp      NotReady   control-plane   1m    v1.31.x
```

> The node shows **NotReady** because we haven't installed a CNI plugin yet. That's next!

kubectl Access from Your Local Machine

1. Copy the kubeconfig to your local machine

```
# from your local machine
scp root@<control-plane-ip>:/etc/kubernetes/admin.conf ~/.kube/config
```

2. Find the hostname the API server certificate expects

```
# check the Subject Alternative Names (SANs) in the API server cert
ssh root@<control-plane-ip>

openssl x509 -in /etc/kubernetes/pki/apiserver.crt -text -noout | grep -A1 "Subject Alternative Name"
```

The output will show the DNS names and IPs the cert is valid for (e.g. `kubernetes`, `kubernetes.default`, the node hostname).

3. Update your local hosts file

```
# add an entry so the hostname in the kubeconfig resolves to the control plane IP
echo "<control-plane-ip>  cp" | sudo tee -a /etc/hosts
```

Now `kubectl get nodes` works from your laptop, authenticated over TLS.

Joining Worker Nodes

On each **worker node**, run the join command from `kubeadm init`:

```
sudo kubeadm join <control-plane-ip>:6443 \
  --token <token> \
  --discovery-token-ca-cert-hash sha256:<hash>
```

If you lost the token, generate a new one from the control plane:

```
kubeadm token create --print-join-command
```

Verify from the Control Plane

```
kubectl get nodes
```

```
NAME      STATUS     ROLES           AGE   VERSION
cp        NotReady   control-plane   5m    v1.34.x
worker    NotReady   <none>          1m    v1.34.x
```

Installing a CNI Plugin – Calico

A **Container Network Interface (CNI)** plugin provides pod networking.

- Each Pod needs its own IP address
- Pods must communicate across nodes without NAT
- The Kubernetes networking model **requires** a CNI plugin

Install Calico

```
# Download the manifest
wget https://raw.githubusercontent.com/projectcalico/calico/v3.28.0/manifests/calico.yaml

# Edit the CALICO_IPV4POOL_CIDR
sed -i 's|# - name: CALICO_IPV4POOL_CIDR|- name: CALICO_IPV4POOL_CIDR|' calico.yaml
sed -i 's|# value: "192.168.0.0/16"| value: "10.100.0.0/16"|' calico.yaml

# Apply
kubectl apply -f calico.yaml
```

Watch Nodes Become Ready

```
kubectl get nodes -w
```

Verifying the Cluster

Check Node Status

```
kubectl get nodes -o wide
```

Check System Pods

```
kubectl get pods -n kube-system
```

You should see:

- `coredns-*` – DNS for the cluster
- `etcd-cp` – the key-value store
- `kube-apiserver-cp` – the API server
- `kube-controller-manager-cp` – the controller manager
- `kube-scheduler-cp` – the scheduler
- `kube-proxy-*` – one on each node
- `calico-node-*` – one on each node (CNI)

Setting bash autocomplete and alias

```
apt update && apt install -y bash-completion
echo 'source <(kubectl completion bash)' >> ~/.bashrc
echo 'source /usr/share/bash-completion/bash_completion' >> ~/.bashrc
echo 'alias k=kubectl' >> ~/.bashrc
echo 'complete -o default -F __start_kubectl k' >> ~/.bashrc
source ~/.bashrc
```

Inspecting Core Components

Static Pods on the Control Plane

```
ls /etc/kubernetes/manifests/
```

```
etcd.yaml  kube-apiserver.yaml  kube-controller-manager.yaml  kube-scheduler.yaml
```

These are **static Pods** — managed directly by the kubelet, not the API server.

Inspect etcd

```
kubectl -n kube-system exec etcd-cp -- etcdctl \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  member list
```

Install etcdctl on the Host

Instead of exec-ing into the etcd container, install `etcdctl` directly on the control plane node.

1. Download the etcd release

```
ETCD_VERSION="v3.5.21"

curl -fsSL https://github.com/etcd-io/etcd/releases/download/${ETCD_VERSION}/etcd-${ETCD_VERSION}-linux-amd64.tar.gz \
  | sudo tar -xz --strip-components=1 -C /usr/local/bin/ etcd-${ETCD_VERSION}-linux-amd64/etcdctl

# etcdctl defaults to API v2; Kubernetes uses v3, so we must set this to interact with the cluster's data
export ETCDCTL_API=3
```

2. Verify

```
etcdctl version
```

Take an etcd Snapshot

3. Save a snapshot from the host

```
sudo etcdctl snapshot save /tmp/etcd-backup.db \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key
```

4. Verify the snapshot

```
sudo etcdctl snapshot status /tmp/etcd-backup.db --write-out=table
```

Why from the host? Running `etcdctl` on the host (instead of `kubectl exec`) means backups work even if the API server is down — exactly when you need them most.

Inspecting Core Components (cont.)

Check the API Server

```
# /healthz is the API server's health check endpoint — returns "ok" if the server is able to handle requests
kubectl get --raw /healthz
kubectl get --raw /version
```

Check Kubelet Status

```
sudo systemctl status kubelet

sudo journalctl -u kubelet --no-pager -l | tail -20
```

Check the Controller Manager and Scheduler

```
kubectl get componentstatuses   # deprecated but still works
kubectl get --raw /readyz?verbose
```

> The **kubelet** is the only component that runs as a systemd service. Everything else on the control plane runs as a static Pod.

Examine Certificates

The cluster PKI lives under `/etc/kubernetes/pki/` on the control plane node.

List All Certificates

```
# show every certificate and its expiration date
sudo kubeadm certs check-expiration
```

Inspect a Specific Certificate

```
# examine the API server's serving certificate
openssl x509 -in /etc/kubernetes/pki/apiserver.crt -text -noout | \
  grep -A2 "Validity"
```

Examine certificates (cond.)

Key Files to Know

File	Purpose
`ca.crt` / `ca.key`	Cluster CA – signs all other certs
`apiserver.crt`	API server serving certificate
`apiserver-kubelet-client.crt`	API server → kubelet client cert
`front-proxy-ca.crt`	CA for the aggregation layer
`etcd/ca.crt`	Separate CA for etcd communication
`sa.key` / `sa.pub`	Key pair for signing ServiceAccount tokens

All certificates generated by kubeadm are valid for **1 year** by default. Use `kubeadm certs renew all` to renew them before they expire.

Create a New User

Kubernetes has no "user" object — users are identified by certificates signed by the cluster CA.

1. Generate a Private Key and CSR

```
# create a private key for the new user
openssl genrsa -out jane.key 2048

# create a certificate signing request (CN = username, O = group)
openssl req -new -key jane.key -out jane.csr -subj "/CN=jane/O=dev-team"
```

2. Sign the Certificate with the Cluster CA

```
sudo openssl x509 -req -in jane.csr \
  -CA /etc/kubernetes/pki/ca.crt \
  -CAkey /etc/kubernetes/pki/ca.key \
  -CAcreateserial \
  -out jane.crt -days 365
```

Create a New User (cont.)

3. Add Credentials to kubeconfig

```
# set the user credentials
kubectl config set-credentials jane \
  --client-certificate=jane.crt \
  --client-key=jane.key

# create a context for the new user
kubectl config set-context jane-context \
  --cluster=kubernetes \
  --namespace=default \
  --user=jane
```

4. Test the New User

```
# switch to jane's context
kubectl config use-context jane-context

# this will fail — jane has no RBAC permissions yet
kubectl get pods
```

Next step: You'd create a Role and RoleBinding to grant jane access. We'll cover RBAC in detail on Day 3.

Deploy Your First Pod

Create an nginx Pod

```
kubectl run nginx --image=nginx:latest --port=80 --dry-run=client -o yaml > pod.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    run: nginx
  name: nginx
spec:
  containers:
  - image: nginx:latest
    name: nginx
    ports:
    - containerPort: 80
```

```
kubectl apply -f pod.yaml
```

Verify It's Running

```
kubectl get pods -o wide
kubectl describe pod nginx
kubectl logs nginx
```

Deploy Your First Pod (cont.)

Expose It

```
kubectl expose pod nginx --type=NodePort --port=80
kubectl get svc nginx
```

Clean Up

```
kubectl delete pod nginx
kubectl delete svc nginx
```

NOTE: kubectl command cheat sheet

Day 1 Recap

What We Covered Today

- ✅ Containers vs Virtual Machines – the evolution of app deployment
- ✅ Kubernetes Architecture – control plane, worker nodes, and how they interact
- ✅ Component Communication & TLS – mTLS, server/client certificates, cluster PKI
- ✅ Kubernetes Interfaces – CRI, CNI, and CSI plugin standards
- ✅ Installing Kubernetes – kubeadm from scratch on cloud VMs
- ✅ Cluster Validation – verifying nodes, components, and networking

Coming Up Tomorrow

Day 2: Core Concepts and Workload Management

- Managing Kubernetes Objects (Namespaces, Pods, Deployments)
- Scaling Applications
- ConfigMaps and Secrets
- Health Checks: Liveness and Readiness Probes
- Working with kubectl

Labs

- Deploying sample applications
- Rolling updates and rollbacks
- Using ConfigMaps and Secrets
- Pod health monitoring and recovery

Questions?